
Rapport Technique

Service de Localisation GPS

Microservice Node.js/TypeScript — gRPC, TimescaleDB, Kafka et
Géofencing

Système de Gestion de Flotte de Véhicules

Master 1 GIL — 2025/2026

Informations du projet

Auteurs :	Kherbachi Rima & Mouhoubi Leiticia
Formation :	Master 1 GIL
Université :	Université de Rouen Normandie
Encadrants :	Lydia et Luc
Année :	2025 – 2026
Semaine documentée :	5

Mars – Avril 2026

Table des matières

1	Introduction et Architecture	3
1.1	Présentation du service	3
1.2	Structure du projet	3
1.3	Diagramme de flux	4
2	Bibliothèques et Dépendances	5
3	Modèle de Données — TimescaleDB	6
3.1	Contrat gRPC (localisation.proto)	6
3.2	Table TimescaleDB	7
4	Géofencing et Intégration Kafka	8
4.1	Zones définies	8
4.2	Moteur de détection	8
4.3	Publication des alertes Kafka	9
5	API REST — Historique et Health Check	10
5.1	Endpoints disponibles	10
5.2	Simulateur GPS	10
6	Tests Unitaires Jest	11
6.1	Stratégie de tests	11
6.2	Exemples de cas de tests	11
6.3	Résultat de l'exécution des tests	12
7	Tests des Services et API	14
7.1	Tests de l'API REST (Service Localisation)	14
7.1.1	Health Check	14
7.1.2	Récupération des zones	14
7.2	Tests de l'API GraphQL (Apollo Gateway)	15
7.2.1	Création et gestion des données	15
7.2.2	Consultation de l'historique et dernière position	16
7.3	Tests des communications gRPC	17
8	Déploiement et Observabilité	18
8.1	Variables d'environnement	18
8.2	Observabilité OpenTelemetry	18
8.3	Kubernetes	18
8.4	Difficultés rencontrées et solutions	19

Conclusion

20

Chapitre 1

Introduction et Architecture

1.1 Présentation du service

La semaine 5 est consacrée au développement du **service de localisation GPS**, le composant le plus complexe du système. Il assure le suivi temps réel de chaque véhicule de la flotte en combinant quatre technologies distinctes :

lightblue Technologie	Port	Rôle
gRPC (@grpc/grpc-js)	50051	Streaming bidirectionnel des positions GPS + requêtes historiques
HTTP/REST (Express)	3002	Consultation de l'historique et health check
TimescaleDB (PostgreSQL)	5432	Stockage time-series avec hypertable et indexation temporelle
Apache Kafka	9092	Publication des alertes de géofencing
Prometheus (OpenTelemetry)	9465	Métriques et traces distribuées

TABLE 1.1 – Interfaces du service localisation

1.2 Structure du projet

```
1 localisation-service/  
2   src/  
3     main.ts                # Orchestration : gRPC + HTTP +  
4       Simulateur  
5   proto/  
6     localisation.proto     # Contrat gRPC (Position, Ack,  
7       Historique)  
8   database/  
9     timescale.ts           # Connexion TimescaleDB, hypertable,  
10    requetes  
11   kafka/  
12     producer.ts           # Publication alertes geofencing  
13   geofencing/
```

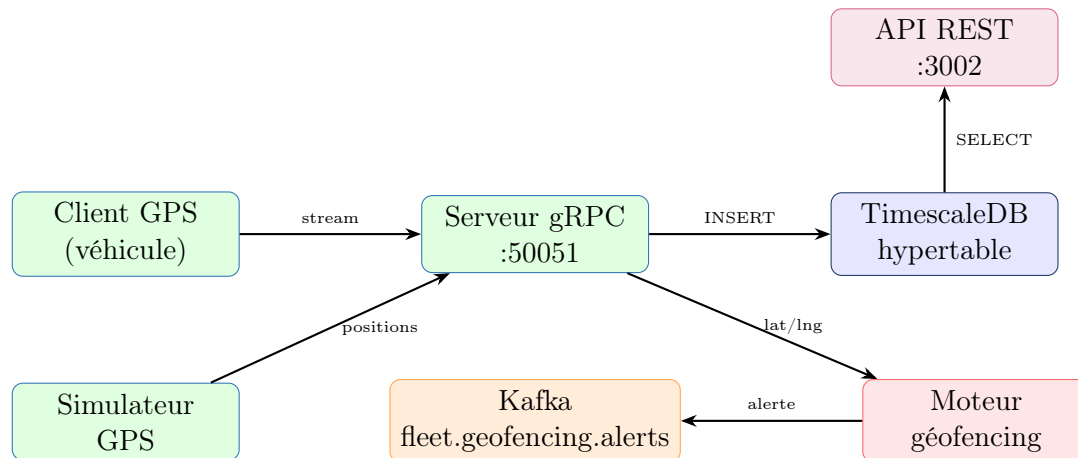
```

11     zones.ts                # Zones autorisees/interdites +
        detection
12 http/
13     server.ts              # API REST Express (historique, last
        position)
14     simulator/
15         gps-simulator.ts    # Simulation de donnees GPS (5
        vehicules)
16     telemetry/
17         tracing.ts          # OpenTelemetry : Jaeger + Prometheus
        :9465
18     tests/
19         geofencing.spec.ts  # 7 tests unitaires du moteur de
        geofencing
20         gps-simulator.spec.ts # 5 tests unitaires du simulateur GPS
21 Dockerfile                # Build multi-stage, utilisateur non-
        root
22 package.json
23 tsconfig.json

```

Listing 1.1 – Arborescence du service localisation

1.3 Diagramme de flux



Chapitre 2

Bibliothèques et Dépendances

lightblue thèque	Biblio-	Package npm	Rôle et Justification
gRPC		@grpc/grpc-js, @grpc/proto-loader	Serveur gRPC bidirectionnel, chargement dynamique du fichier <code>.proto</code> .
Base de données		pg	Client PostgreSQL/TimescaleDB natif pour les INSERT et SELECT time-series.
Kafka		kafkajs	Client Kafka TypeScript pour publier les alertes de géofencing sur <code>fleet.geofencing.alerts</code> .
API REST		express	Serveur HTTP léger pour l'historique et le health check.
Observabilité		@opentelemetry/sdk-node, @opentelemetry/exporter-jaeger, @opentelemetry/exporter-prometheus	Traces Jaeger + métriques Prometheus sur le port 9465.
Tests		jest, ts-jest, @types/jest	Tests unitaires TypeScript sans dépendance externe.

TABLE 2.1 – Dépendances du localisation-service (`package.json`)

Chapitre 3

Modèle de Données — TimescaleDB

3.1 Contrat gRPC (localisation.proto)

Le fichier `.proto` définit deux méthodes RPC : un flux bidirectionnel pour la réception continue des positions et une méthode unaire pour les requêtes historiques.

```
1 syntax = "proto3";
2 package localisation;
3
4 service LocalisationService {
5     rpc StreamPositions (stream Position) returns (stream Ack) {}
6     rpc GetHistorique (HistoriqueRequest) returns (HistoriqueResponse) {}
7 }
8
9 message Position {
10     string vehicule_id = 1;
11     double latitude     = 2;
12     double longitude    = 3;
13     double vitesse      = 4;
14     string horodatage    = 5;    // ISO 8601
15 }
16
17 message Ack {
18     bool    success = 1;
19     string  message = 2;
20 }
21
22 message HistoriqueRequest {
23     string vehicule_id = 1;
24     string depuis      = 2;    // ISO 8601
25     string jusqu_a     = 3;    // ISO 8601
26 }
27
28 message HistoriqueResponse {
29     repeated Position positions = 1;
30 }
```

Listing 3.1 – localisation.proto – contrat du service

3.2 Table TimescaleDB

TimescaleDB est une extension PostgreSQL optimisée pour les données temporelles. L'initialisation convertit la table **positions** en **hypertable** (partitionnement automatique par intervalles de temps) et crée un index composé pour les requêtes par véhicule et plage temporelle.

```
1 CREATE TABLE IF NOT EXISTS positions (  
2     vehicule_id TEXT NOT NULL,  
3     latitude DOUBLE PRECISION NOT NULL,  
4     longitude DOUBLE PRECISION NOT NULL,  
5     vitesse DOUBLE PRECISION NOT NULL,  
6     horodatage TIMESTAMPTZ NOT NULL DEFAULT NOW()  
7 );  
8  
9 -- Conversion en hypertable (partitionnement temporel automatique)  
10 SELECT create_hypertable('positions', 'horodatage',  
11                          if_not_exists => TRUE);  
12  
13 -- Index compose : requetes par vehicule sur une plage de temps  
14 CREATE INDEX IF NOT EXISTS idx_positions_vehicule_temps  
15 ON positions (vehicule_id, horodatage DESC);
```

Listing 3.2 – Création de la table et hypertable (timescale.ts)

Chapitre 4

Géofencing et Intégration Kafka

4.1 Zones définies

Trois zones géographiques sont définies autour de Rouen à des fins de test. La détection repose sur un modèle de boîte englobante (*bounding box*).

lightblue Zone	Type	Coordonnées (lat/lng)
zone-rouen-centre	AUTORISEE	lat [49.43, 49.46], lng [1.07, 1.12]
zone-industrielle-interdite	INTERDITE	lat [49.40, 49.43], lng [1.07, 1.12]
zone-port-interdite	INTERDITE	lat [49.42, 49.45], lng [1.02, 1.07]

TABLE 4.1 – Zones de géofencing autour de Rouen

4.2 Moteur de détection

```
1 export function checkGeofencing(  
2   lat: number, lng: number  
3 ): GeofencingResult {  
4   for (const zone of ZONES) {  
5     if (zone.type === 'INTERDITE' && isInZone(lat, lng, zone)) {  
6       return { violated: true, zone };  
7     }  
8   }  
9   return { violated: false };  
10 }  
11  
12 export function isInZone(lat: number, lng: number, zone: Zone):  
13   boolean {  
14   return lat >= zone.minLat && lat <= zone.maxLat  
15     && lng >= zone.minLng && lng <= zone.maxLng;  
16 }
```

Listing 4.1 – Détection de violation (zones.ts)

4.3 Publication des alertes Kafka

En cas de violation, un événement `GeofencingAlert` est publié sur le topic `fleet.geofencing.alert`. La connexion est non bloquante : si Kafka est indisponible au démarrage, le service continue de fonctionner normalement (les alertes sont silencieusement ignorées).

```
1 await producer.send({
2   topic: KAFKA_TOPIC_GEOFENCING,
3   messages: [{
4     key: alert.vehiculeId,
5     value: JSON.stringify({
6       vehiculeId: alert.vehiculeId,
7       latitude: alert.latitude,
8       longitude: alert.longitude,
9       zoneId: alert.zoneId,
10      zoneName: alert.zoneName,
11      type: alert.type, // 'ENTREE' / 'SORTIE'
12      timestamp: alert.timestamp,
13    }),
14  }],
15 });
```

Listing 4.2 – Publication d’une alerte (kafka/producer.ts)

Chapitre 5

API REST — Historique et Health Check

5.1 Endpoints disponibles

Un serveur Express écoute sur le port 3002 et expose les endpoints suivants :

lightblue Méth.	URL	Description
GET	/health	Santé du service (200 OK)
GET	/api/positions/:id/historique?depuis=&jusqu_a=	Historique sur une plage temporelle (ISO 8601)
GET	/api/positions/:id/last	Dernière position connue du véhicule

TABLE 5.1 – Endpoints REST du localisation-service

5.2 Simulateur GPS

Pour les tests, un simulateur génère des positions réalistes pour 5 véhicules (VEH-001 à VEH-005) en appliquant une dérive progressive à partir du centre de Rouen (49.443232°N, 1.099971°E).

```
1 // Dérive aleatoire de +/- 0.002 degres (~200 m) par tick
2 const newLat = current.lat + (Math.random() - 0.5) * 0.004;
3 const newLng = current.lng + (Math.random() - 0.5) * 0.004;
4
5 return {
6   vehicule_id: vehiculeId,
7   latitude:    Math.round(newLat * 1_000_000) / 1_000_000,
8   longitude:   Math.round(newLng * 1_000_000) / 1_000_000,
9   vitesse:     Math.round(Math.random() * 130),
10  horodatage:  new Date().toISOString(),
11 };
```

Listing 5.1 – Dérive progressive des positions (gps-simulator.ts)

Le simulateur est activé via la variable d'environnement `ENABLE_SIMULATOR=true`.

Chapitre 6

Tests Unitaires Jest

6.1 Stratégie de tests

Deux suites de tests Jest couvrent la logique métier pure, sans aucune dépendance externe (pas de base de données, pas de Kafka) :

lightblue Fichier test	Nb tests	Scénarios couverts
geofencing.spec.ts	7	isInZone, checkGeofencing, getZonesForPosition
gps-simulator.spec.ts	5	generatePosition, startSimulator
Total	12	

TABLE 6.1 – Couverture des tests unitaires Jest du localisation-service

6.2 Exemples de cas de tests

```
1 it('detecte une violation dans la zone industrielle interdite', ()  
  => {  
2   const result = checkGeofencing(49.42, 1.09);  
3   expect(result.violated).toBe(true);  
4   expect(result.zone!.type).toBe('INTERDITE');  
5   expect(result.zone!.id).toBe('zone-industrielle-interdite');  
6 });  
7  
8 it('ne detecte pas de violation dans une zone autorisee', () => {  
9   // Centre de Rouen -- zone autorisee  
10  const result = checkGeofencing(49.44, 1.09);  
11  expect(result.violated).toBe(false);  
12  expect(result.zone).toBeUndefined();  
13 });  
14  
15 it('detecte une violation dans la zone port interdite', () => {  
16  const result = checkGeofencing(49.43, 1.05);  
17  expect(result.violated).toBe(true);  
18  expect(result.zone!.id).toBe('zone-port-interdite');
```

```
19 });  
20  
21 it('retourne true sur les limites exactes de la zone', () => {  
22   expect(isInZone(zone.minLat, zone.minLng, zone)).toBe(true);  
23   expect(isInZone(zone.maxLat, zone.maxLng, zone)).toBe(true);  
24 });
```

Listing 6.1 – Tests du moteur de géofencing (geofencing.spec.ts)

```
1 it('genere une vitesse comprise entre 0 et 130 km/h', () => {  
2   for (let i = 0; i < 50; i++) {  
3     const { vitesse } = generatePosition('VEH-002');  
4     expect(vitesse).toBeGreaterThanOrEqual(0);  
5     expect(vitesse).toBeLessThanOrEqual(130);  
6   }  
7 });  
8  
9 it('genere un horodatage ISO 8601 valide', () => {  
10   const { horodatage } = generatePosition('VEH-003');  
11   expect(new Date(horodatage).toISOString()).toBe(horodatage);  
12 });  
13  
14 it('appelle le callback pour chaque tick', async () => {  
15   const received: string[] = [];  
16   const timer = startSimulator((pos) => {  
17     received.push(pos.vehicule_id);  
18   }, 50);  
19   await new Promise((resolve) => setTimeout(resolve, 120));  
20   clearInterval(timer);  
21   expect(received.length).toBeGreaterThanOrEqual(10); // 2 ticks x  
22   5 vehicules  
});
```

Listing 6.2 – Tests du simulateur GPS (gps-simulator.spec.ts)

6.3 Résultat de l'exécution des tests

La commande `npx jest -verbose` dans le dossier `services/localisation-service` produit le rapport suivant :

```
PS C:\Systeme-de-Gestion-de-Flotte-de-Vehicules\services\localisation-service> npm run test:coverage
> localisation-service@1.0.0 test:coverage
> jest --coverage

PASS src/tests/geofencing.spec.ts (15.885 s)
PASS src/tests/gps-simulator.spec.ts (16.076 s)
  ● Console

    console.log
      Simulateur GPS démarré – 5 véhicules, intervalle 50ms

    at log (src/simulator/gps-simulator.ts:60:11)

-----|-----|-----|-----|-----|-----
File    | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
-----|-----|-----|-----|-----|-----
All files | 96.66   | 83.33    | 100     | 96.55   |
```

FIGURE 6.1 – Rapport détaillé de l'exécution des 12 tests unitaires via Jest

Chapitre 7

Tests des Services et API

7.1 Tests de l'API REST (Service Localisation)

Le service de localisation, démarré sur le port 3002, expose des endpoints pour vérifier l'état du service et gérer les zones géographiques.

7.1.1 Health Check

L'endpoint `/health` permet de s'assurer que le service est opérationnel.

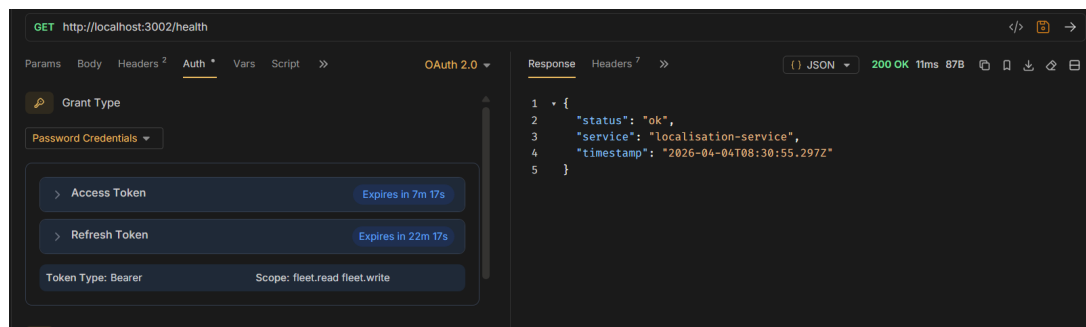


FIGURE 7.1 – Vérification de l'état (Health Check) du service de localisation via Bruno

7.1.2 Récupération des zones

Cet endpoint retourne la liste des zones configurées (autorisées ou interdites) en base de données.

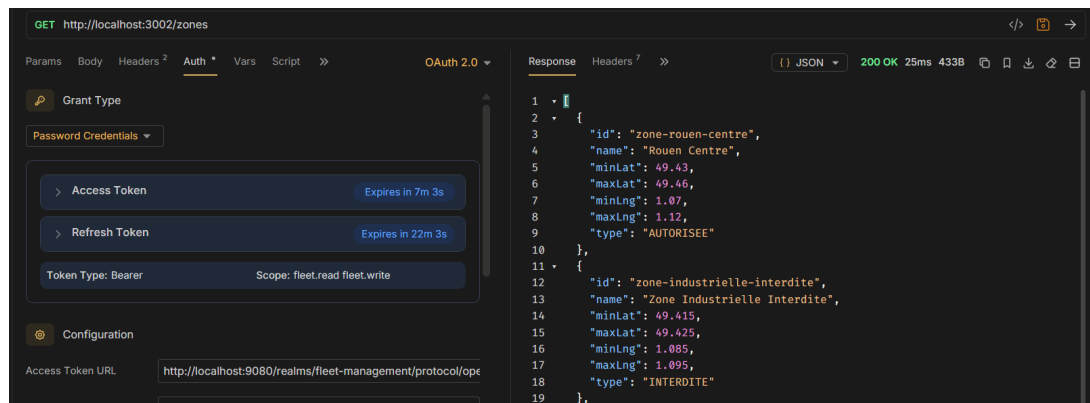


FIGURE 7.2 – Liste des zones géographiques récupérées au format JSON via l'API REST

7.2 Tests de l'API GraphQL (Apollo Gateway)

La Gateway (port 4000) permet de manipuler les données de la flotte via des mutations et des requêtes GraphQL.

7.2.1 Création et gestion des données

Nous avons testé l'insertion de nouvelles positions et la création de zones de parking via l'interface Apollo Studio.

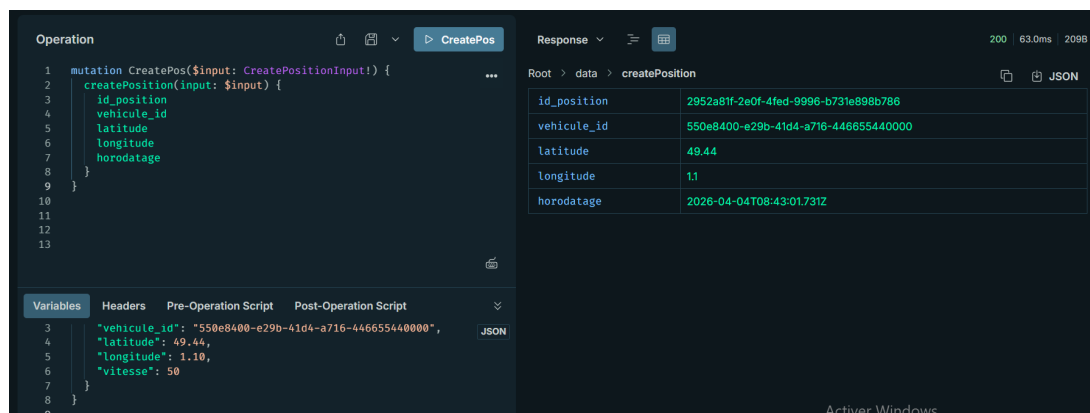


FIGURE 7.3 – Mutation GraphQL CreatePos : Enregistrement réussi d'une nouvelle position

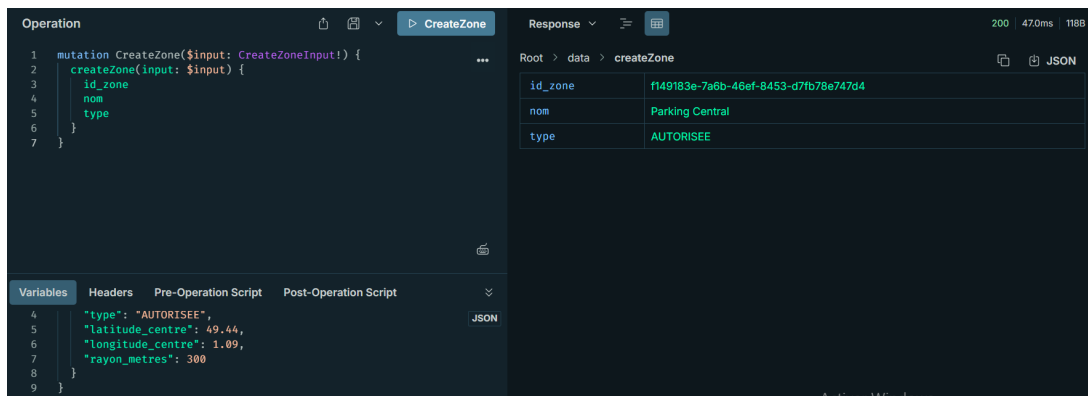


FIGURE 7.4 – Mutation GraphQL `createZone` : Définition d’une nouvelle zone de parking autorisée

7.2.2 Consultation de l’historique et dernière position

Les tests suivants valident la récupération des données de déplacement, soit via Apollo, soit par des requêtes POST directes.

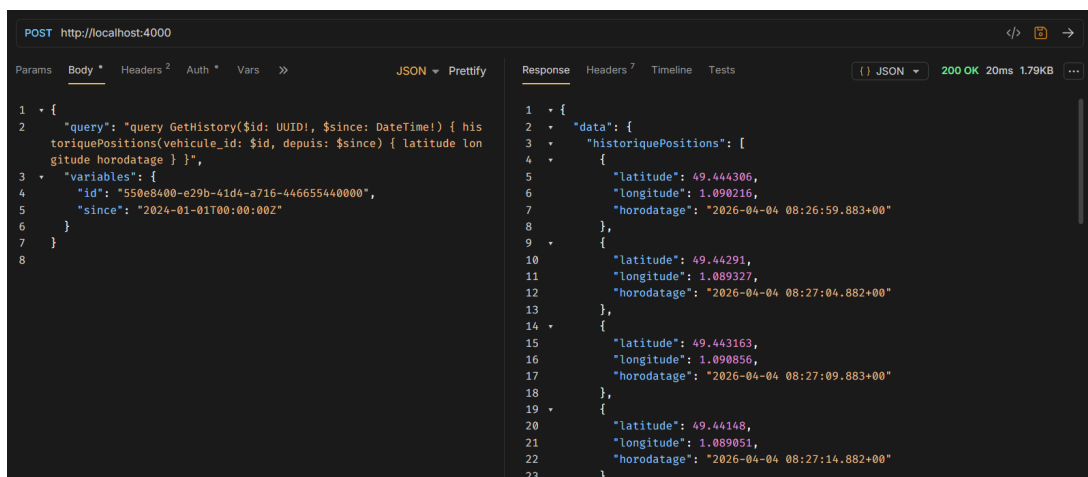


FIGURE 7.5 – Récupération de l’historique des positions via une requête POST GraphQL avec filtrage par date

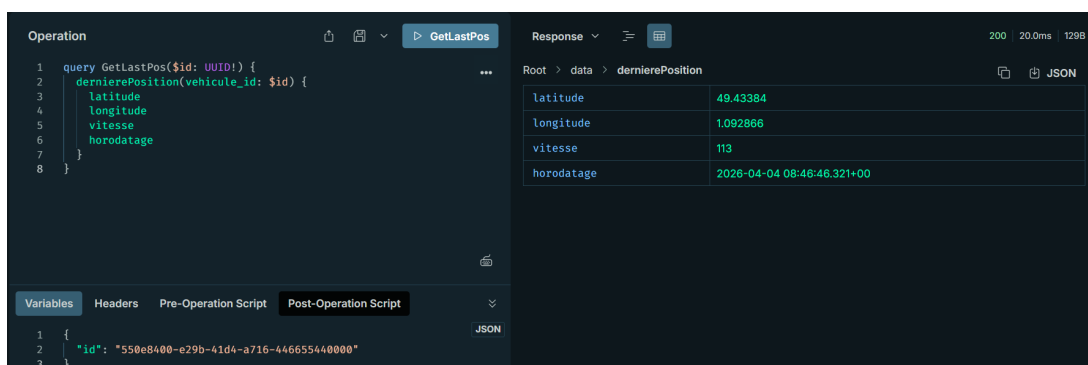


FIGURE 7.6 – Requête `GetLastPos` : Consultation de la dernière position connue du véhicule

7.3 Tests des communications gRPC

Pour valider les échanges de flux de données en temps réel, un script de test gRPC a été exécuté.

```
C:\Users\leiti\Systeme-de-Gestion-de-Flotte-de-Vehicules>npm install @grpc/grpc-js @grpc/proto-loader
added 34 packages in 3s

3 packages are looking for funding
  run 'npm fund' for details

C:\Users\leiti\Systeme-de-Gestion-de-Flotte-de-Vehicules>node scripts/test-grpc.js
--- Test GetHistorique (Unaire) ---
Historique reçu: 423 positions
Dernière position: {
  vehicule_id: '550e8400-e29b-41d4-a716-446655440000',
  latitude: 49.441095,
  longitude: 1.093556,
  vitesse: 108,
  horodatage: '2026-04-04 09:03:40.419+00'
}
--- Test StreamPositions (Bidirectionnel) ---
Envoi position 1...
Reçu du serveur: Position enregistrée
Envoi position 2...
Reçu du serveur: Position enregistrée
Envoi position 3...
Reçu du serveur: Position enregistrée
Stream terminé par le serveur
```

FIGURE 7.7 – Logs d'exécution du client gRPC : validation de l'appel Unaire et du Streaming bidirectionnel

Chapitre 8

Déploiement et Observabilité

8.1 Variables d'environnement

lightblue Variable	Valeur par défaut	Requis
DATABASE_URL	postgresql://admin:...-@timescaledb:5432/localisation_db	Oui
KAFKA_BROKER	localhost:9092	Oui
KAFKA_TOPIC_GEOFENCING	fleet.geofencing.alerts	Non
GRPC_PORT	50051	Non
HTTP_PORT	3002	Non
JAeger_ENDPOINT	http://jaeger:14268/api/traces	Non
ENABLE_SIMULATOR	false	Non

TABLE 8.1 – Variables d'environnement du localisation-service

8.2 Observabilité OpenTelemetry

lightblue Pilier	Outil	Ce qui est mesuré
Traces	Jaeger (port 14268)	Durée de <code>StreamPositions.onData</code> et <code>GetHistorique</code>
Métriques	Prometheus (port 9465)	Taux de positions recues, alertes géofencing
Logs	Console JSON	Connexions, positions, alertes, erreurs

TABLE 8.2 – Piliers d'observabilité du localisation-service

8.3 Kubernetes

- **Deployment** : 2 répliques, `containerPort`: 50051
- **Service** : type `ClusterIP`, port gRPC 50051
- **Ressources** : limits 1000m CPU / 1Gi RAM
- **Liveness probe** : `tcpSocket` sur le port 50051

8.4 Difficultés rencontrées et solutions

lightblue Difficulté	Solution apportée
Fichiers <code>.proto</code> absents dans <code>dist/</code>	Script <code>build</code> mis à jour avec <code>xcopy</code> pour copier les fichiers non-TypeScript
Kafka indisponible au démarrage	Connexion non bloquante avec <code>try/catch</code> – le service continue sans alertes géofencing
Création hypertexte si elle existe déjà	Paramètre <code>if_not_exists => TRUE</code> dans <code>create_hypertext</code>
Conflit de port Prometheus avec conducteur-service	Port 9465 attribué au localisation-service (vs 9464)
Message d'erreur vide au démarrage	<code>err.message ?? err</code> pour couvrir les erreurs sans propriété <code>message</code>

TABLE 8.3 – Difficultés rencontrées et solutions apportées

Conclusion

La semaine 5 marque la livraison du **service de localisation GPS**, composant le plus ambitieux du projet, combinant streaming gRPC, base de données time-series, géofencing et observabilité.

Réalisations principales :

- Serveur gRPC bidirectionnel (**StreamPositions**) avec persistance TimescaleDB en temps réel.
- Méthode gRPC unaire (**GetHistorique**) pour les requêtes historiques.
- Moteur de géofencing avec 3 zones autour de Rouen et publication Kafka des alertes.
- API REST Express pour la consultation de l'historique.
- Simulateur GPS pour 5 véhicules avec dérive progressive.
- Instrumentation OpenTelemetry complète (traces Jaeger + métriques Prometheus).
- **12 tests unitaires Jest** couvrant la logique métier pure (géofencing et simulateur).

Avancement global du projet : 5/7 semaines — 71 %